

# Practica 1

## Sockets

Desarrollo de Aplicaciones Distribuidas I

---

*Grado en ingeniería informática*

*Isabel S. Piñana Alonso*

*Santiago Colomer Andreu*

*Curso académico: 2025/2026*



## Índice de contenidos

|                                                    |          |
|----------------------------------------------------|----------|
| <b>1. Funcionalidad básica</b>                     | <b>2</b> |
| <b>2. Arquitectura general</b>                     | <b>2</b> |
| <b>3. Cliente</b>                                  | <b>3</b> |
| 3.1 Paquete: edu.ucam.cliente                      | 3        |
| ClienteERP                                         | 3        |
| ClientePrincipal                                   | 3        |
| 3.2 Paquete: edu.ucam.cliente.config               | 3        |
| ClientConfig                                       | 3        |
| 3.3 Paquete: edu.ucam.cliente.interfaces           | 4        |
| 3.4 Paquete: edu.ucam.cliente.service              | 4        |
| AuthenticationService                              | 4        |
| ChannelData                                        | 4        |
| CommunicationSocket                                | 4        |
| ResponseParser                                     | 4        |
| 3.5 Paquete: edu.ucam.cliente.service.repositories | 5        |
| BaseRepository                                     | 5        |
| SubjectRepository, TituRepository, MatRepository   | 5        |
| 3.6 Paquete: edu.ucam.cliente.UI                   | 5        |
| <b>4. Servidor</b>                                 | <b>6</b> |
| 4.1 ServidorMain                                   | 6        |
| 4.2 Paquete: edu.ucam.servidor.channels            | 6        |
| MainChannel                                        | 6        |
| ClientChannel                                      | 6        |
| DataChannel                                        | 6        |
| 4.3 Paquete: edu.ucam.servidor.commandHandlers     | 7        |
| 4.4 Paquete: edu.ucam.servidor.repositories        | 7        |
| 4.5 ServerState                                    | 7        |
| <b>5. Protocolo de comunicación</b>                | <b>8</b> |
| <b>6. Flujo completo de ejecución</b>              | <b>8</b> |

## 1. Funcionalidad básica

El proyecto implementa un sistema distribuido basado en sockets TCP, donde un cliente se conecta a un servidor para gestionar datos académicos: Titulaciones, Asignaturas y Matrículas.

El servidor mantiene los datos en memoria y expone un conjunto de comandos que el cliente puede ejecutar. El cliente, por su parte, ofrece una interfaz gráfica y un sistema de repositorios remotos que encapsulan la comunicación.

La comunicación se realiza mediante:

- **Canal principal:** comandos de texto (ADD, GET, LIST, UPDATE, REMOVE, COUNT, SAVE, SESIONES, etc.).
- **Canal secundario:** transferencia de objetos serializados cuando el servidor responde con PREOK.

El sistema está dividido en dos proyectos independientes:

- **Cliente:** interfaz, lógica de comandos, repositorios remotos.
- **Servidor:** gestión de datos, handlers de comandos, control de sesiones.

## 2. Arquitectura general

La arquitectura sigue un modelo **cliente-servidor clásico**, con separación clara de responsabilidades:

### Cliente

- Interfaz gráfica (Swing)
- Comunicación con el servidor
- Repositorios remotos
- Interpretación de comandos
- Transferencia de objetos

### Servidor

- Socket principal de escucha
- Un canal por cliente
- Handlers para cada comando
- Repositorios locales en memoria
- Control de sesiones activas

### Protocolo

El servidor responde siempre con uno de estos tipos:

- OK = operación correcta
- FAILED = error

- PREOK = se abrirá un canal secundario para enviar/recibir objetos

### 3. Cliente

El cliente está compuesto por varios paquetes que separan la lógica de comunicación, repositorios, servicios y UI.

#### 3.1 Paquete: *edu.ucam.cliente*

##### *ClienteERP*

Es la clase principal del cliente y actúa como controlador de comandos.

Sus responsabilidades:

- Mantener la conexión con el servidor (ICommunicationServer)
- Autenticar al usuario (IAuthentication)
- Gestionar repositorios remotos
- Interpretar comandos escritos por el usuario
- Construir la respuesta final para la interfaz

Incluye métodos como:

- gestionarAdd
- gestionarGet
- gestionarList
- gestionarRemove
- gestionarUpdate
- gestionarCount
- imprimirSesiones
- guardarInformacion

Es la clase que orquesta todo el funcionamiento del cliente.

##### *ClientePrincipal*

Contiene el método main del cliente.

Su única función es lanzar la interfaz gráfica inicial:

```
new AuthenticationForm();
```

#### 3.2 Paquete: *edu.ucam.cliente.config*

##### *ClientConfig*

Define los parámetros de conexión:

- IP del servidor
- Puerto del servidor

Es utilizado por CommunicationSocket.

### **3.3 Paquete: *edu.ucam.cliente.interfaces***

Define las interfaces que estructuran el cliente:

- **IAuthentication**: login y cierre de sesión
- **IChannelData**: envío/recepción de objetos
- **ICommunicationServer**: canal principal
- **IRepository**: repositorios remotos genéricos

Estas interfaces permiten desacoplar la lógica del cliente de la implementación concreta.

### **3.4 Paquete: *edu.ucam.cliente.service***

#### ***AuthenticationService***

Implementa la autenticación:

1. Envía USER usuario
2. Si OK = envía PASS contraseña
3. Devuelve true/false

También implementa cerrarSesion() enviando EXIT.

#### ***ChannelData***

Gestiona el canal secundario para objetos.

- Abre un socket temporal a la IP/puerto indicados por el servidor.
- Envía o recibe objetos mediante ObjectOutputStream y ObjectInputStream.

#### ***CommunicationSocket***

Gestiona el socket principal:

- Conecta al servidor
- Envía comandos
- Recibe respuestas
- Mantiene el contador de comandos (idComunicacion)

#### ***ResponseParser***

Interpreta respuestas del servidor:

- OK num codigo mensaje
- FAILED num codigo mensaje
- PREOK num codigo ip puerto

Extrae:

- tipo
- id
- código
- mensaje o ip/puerto

### **3.5 Paquete: *edu.ucam.cliente.service.repositories***

#### ***BaseRepository***

Clase abstracta que implementa la lógica común de los repositorios remotos:

- add
- delete
- list
- update
- getModel
- modelSize

Implementa el protocolo:

1. Enviar comando
2. Si PREOK = abrir canal secundario
3. Enviar/recibir objeto
4. Esperar OK final

#### ***SubjectRepository, TituRepository, MatRepository***

Extienden BaseRepository y definen:

- Comandos específicos (ADDASIG, LISTTIT, etc.)
- Cómo crear objetos (crearObjeto)

### **3.6 Paquete: *edu.ucam.cliente.UI***

Incluye la interfaz gráfica:

- **AuthenticationForm**: login
- **MainUI**: interfaz principal
- **UISelection**: menú de selección

Estas clases permiten al usuario interactuar con el sistema sin usar consola.

## 4. Servidor

El servidor es responsable de:

- Aceptar conexiones
- Interpretar comandos
- Gestionar repositorios locales
- Controlar sesiones
- Coordinar canales de datos

### 4.1 *ServidorMain*

Es el punto de entrada del servidor:

- Crea el socket de escucha
- Acepta clientes
- Crea un ClientChannel por cliente
- Mantiene el servidor activo hasta que se cierre manualmente

### 4.2 *Paquete: edu.ucam.servidor.channels*

#### *MainChannel*

Gestiona el socket principal del servidor:

- Acepta conexiones entrantes
- Crea un hilo ClientChannel por cliente

#### *ClientChannel*

Gestiona la comunicación con un cliente concreto:

- Lee comandos
- Extrae id y operación
- Llama al handler correspondiente
- Envía respuestas OK/FAILED/PREOK

#### *DataChannel*

Canal secundario para objetos:

- Abre un socket temporal
- Envía PREOK ip puerto
- Espera conexión del cliente
- Envía/recibe objetos serializados

### **4.3 Paquete: *edu.ucam.servidor.commandHandlers***

Cada handler implementa la lógica de un comando:

- AddHandler
- GetHandler
- ListHandler
- RemoveHandler
- UpdateHandler
- CountHandler

Todos siguen el mismo patrón:

1. Validar comando
2. Acceder al repositorio
3. Enviar respuesta o abrir canal de datos

### **4.4 Paquete: *edu.ucam.servidor.repositories***

Incluye los repositorios locales:

- AsignaturaRepository
- MatriculaRepository
- TitulacionRepository

Todos gestionan datos en memoria mediante listas.

### **4.5 ServerState**

Mantiene:

- Sesiones activas
- Repositorios
- Configuración global



## 5. Protocolo de comunicación

El servidor responde siempre con:

**OK**

Operación correcta.

**FAILED**

Error en la operación.

**PREOK**

Indica que se abrirá un canal secundario para enviar/recibir objetos.

Ejemplo:

PREOK 5 500 127.0.0.1 6000

## 6. Flujo completo de ejecución

1. El cliente se conecta al servidor.
2. El servidor envía mensaje de bienvenida.
3. El cliente envía USER y PASS.
4. El usuario introduce comandos en la interfaz.
5. ClienteERP interpreta el comando.
6. El repositorio remoto envía el comando al servidor.
7. El servidor ejecuta el handler correspondiente.
8. Si es necesario, abre un DataChannel.
9. El cliente envía/recibe objetos.
10. El servidor envía OK final.
11. La interfaz muestra la respuesta.